Compiler Design Laboratory
Assignment VII: Flex and Bison Parser Construction

---

1. Objective

The objective of this assignment is to learn how to construct parsers using the **Bison** parser generator and integrate it with the **Flex** lexical analyzer.

- Understand the role of Bison in syntax analysis
- Understand how Flex and Bison interact
- Compile and execute a Flex+Bison parser
- Parse arithmetic expressions

2. Introduction to Bison

Bison is a parser generator that automatically produces a syntax analyzer from a context-free grammar specification.

- Bison generates **LALR bottom-up parsers**
- Grammar rules are written in files with extension `.y`
- Bison generates a C parser program
- It is commonly used with the **Flex lexical analyzer**

3. Relationship Between Flex and Bison

- Flex performs **Lexical Analysis**
- Bison performs **Syntax Analysis**
- Flex generates the function `yylex()`
- Bison calls `yylex()` to receive tokens
- Tokens produced by Flex are defined in Bison using `%token`

4. Check Availability in Ubuntu

Check Bison:

```
bison --version
```

Check Flex:

```
flex --version
```

5. Installation (If Not Installed)

```
sudo apt update
sudo apt install bison flex
```

6. Structure of a Bison Program

A Bison file has three sections separated by %%

```
%{
C declarations
%}

Bison declarations

%%

Grammar rules

%%

Additional C code
```

7. Sample Bison Grammar (parser.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

%token NUMBER

%%

expr : expr '+' term
     | expr '-' term
     | term
     ;

term : term '*' factor
     | term '/' factor
     | factor
     ;

factor : '(' expr ')'
       | NUMBER
       ;
```

```
%%

int main()
{
    printf("Enter␣expression:\n");
    if(yyparse()==0)
        printf("Parsing␣successful\n");
    return 0;
}

int yyerror(char *s)
{
    printf("Syntax␣Error\n");
    return 0;
}
```

8. Sample Flex Lexer (lexer.l)

```
%{
#include "parser.tab.h"
%}

%%

[0-9]+       { return NUMBER; }
"+"          { return '+'; }
"-"          { return '-'; }
"*"          { return '*'; }
"/"          { return '/'; }
"("          { return '('; }
")"          { return ')'; }

[ \t\n]      ;   /* ignore whitespace */

.            { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

9. Compilation Steps

Run the following commands in terminal:

```
bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser -lfl
```

Generated files:

- parser.tab.c
- parser.tab.h
- lex.yy.c

10. Execute the Parser

```
./parser
```

11. Sample Input Strings

Valid Expressions

```
3+5
4+7*2
(3+4)*5
8*(2+6)
(5+3)*(2+4)
```

Invalid Expressions

```
3++5
4+*7
(3+5
8*(2+)
*5+3
```

12. Expected Output

Valid Input

```
Enter expression:
3+5*2
Parsing successful
```

Invalid Input

```
Enter expression:
3++5
Syntax Error
```

13. Assignment Problem

Students must implement a Flex+Bison parser for arithmetic expressions.

1. Install and verify the availability of **Flex** and **Bison**.
2. Write the following two files:
   - `parser.y` (Bison grammar)
   - `lexer.l` (Flex lexical analyzer)
3. The parser must support:
   - Addition (+)
   - Subtraction (-)
   - Multiplication (*)
   - Division (/)
   - Parentheses
4. Compile the program using the commands:

```
bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser -lfl
```

5. Execute the parser.
6. Test the parser with:
   - At least five valid expressions
   - At least five invalid expressions

Submission Requirements

- parser.y file
- lexer.l file
- compilation commands
- sample inputs and outputs

14. Learning Outcomes

After completing this assignment, students will be able to:

- Understand interaction between Flex and Bison
- Implement lexical analyzers and parsers
- Build simple language parsers
- Compile and run integrated compiler front-end tools

— End of Assignment —